CN Assignment-2 Report

Name: Maida Syed | Roll No: 24K-0918 | Seed: 918

Approach and Implementation

I wrote a small discrete-event simulator in Python (`gbn_sr.py`). It models the satellite link (R = 2 Mbps, RTT = 500 ms, L = 8,000 bits, so L/R = 4 ms and one-way delay = 250 ms). The channel is written by me and can lose, corrupt and delay both data packets and ACKs. Every packet carries a 16-bit Internet checksum, so a corrupted packet or ACK is detected and thrown away. Sequence numbers wrap modulo 2^k, and the receiver only uses the wire sequence number (never the hidden packet index), so wrap-around is tested properly. At the end the program checks that all packets were delivered exactly once and in order. Assumptions: the sender sends one packet every 4 ms (link busy time), ACK transmission time is ignored, timeout = 700 ms (a bit more than RTT + one possible delay), and extra delay is 50-150 ms.

GBN: sender uses `base`, `nextseqnum`, one timer for the oldest unACKed packet, cumulative ACKs, and on timeout resends everything from `base` to `nextseqnum-1`. Receiver keeps `expectedseqnum`, discards out-of-order packets and re-sends the ACK of the last in-order packet.

SR: sender window with `send_base`, a separate timer for every packet, and individual ACKs; on timeout only that packet is resent. Receiver has its own window (`rcv_base`), buffers out-of-order packets, ACKs each packet individually (also re-ACKs old duplicates) and delivers in order when the gap is filled.

Implementation

Implemented in `gbn_sr.py` (see README for parameters and sample command). Configurable: protocol, number of packets, window N, sequence bits k, loss, corruption, delay probabilities and timeout. Loss, corruption and delay apply to packets and ACKs; corruption is detected by the checksum. A log line is printed for every send, receive, ACK, loss, corruption, delay, timeout and retransmission. Features: GBN has `base`/`nextseqnum`, cumulative ACK, one timer, go-back resend, discard of out-of-order packets. SR has sender and receiver windows, individual ACKs, one timer per packet, receiver buffering, selective resend and in-order delivery. Tested with up to 20% loss + 10% corruption + 20% delay: all packets are delivered in order for both protocols.

Experiments and Comparison

The experiments were performed using 200 packets, window size N = 4, k = 3, seed 918, and a timeout of 700 ms. Three scenarios were tested: a reliable channel, a channel with 10% packet loss, and a mixed channel with packet loss, corruption, and delay.

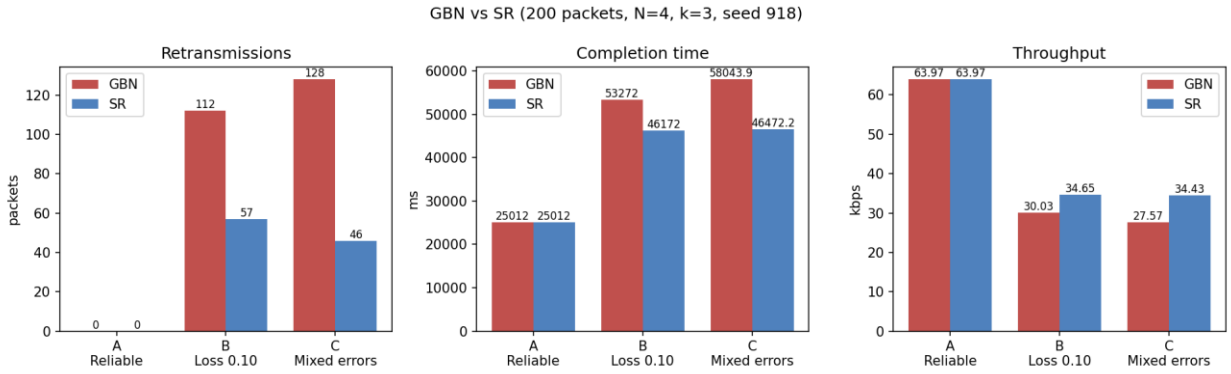
In the reliable channel, both Go-Back-N (GBN) and Selective Repeat (SR) transmitted all 200 packets without retransmissions and achieved the same throughput of 63.97 kbps.

In the packet-loss scenario, GBN required 112 retransmissions, while SR required only 57. The throughput of GBN was 30.03 kbps, compared to 34.65 kbps for SR.

In the mixed-error scenario, GBN required 128 retransmissions, whereas SR required only 46. GBN achieved a throughput of 27.57 kbps, while SR achieved 34.43 kbps.
The average results from 10 runs also showed that SR performed better than GBN, with fewer retransmissions and higher throughput.
GBN retransmits the whole outstanding window after every timeout, including packets that already arrived, because its receiver discards out-of-order packets. SR buffers them and resends only the lost packet. So SR needed fewer retransmissions and had higher throughput under errors. The gap is larger in scenario C because delayed and corrupted packets arrive out of order

Results (TASK 4)

Scenario	Protocol	Packets transmitted	Retransmissions	Delivered correctly	Time (ms)	Throughput (kbps)
A Reliable	GBN	200	0	200	25,012	63.97
A Reliable	SR	200	0	200	25,012	63.97
B Loss 0.10	GBN	312	112	200	53,272	30.03
B Loss 0.10	SR	257	57	200	46,172	34.65
C Mixed	GBN	328	128	200	58,044	27.57
C Mixed	SR	246	46	200	46,472	34.43



Conclusion

Both protocols deliver all packets correctly and in order on a lossy, corrupting and delaying satellite link. On a perfect channel they are identical, but under errors SR uses fewer retransmissions and gives higher throughput, because it buffers out-of-order packets and resends only what is missing. On this slow, long-delay link SR is the better choice, as long as the window is at most 2^(k-1).